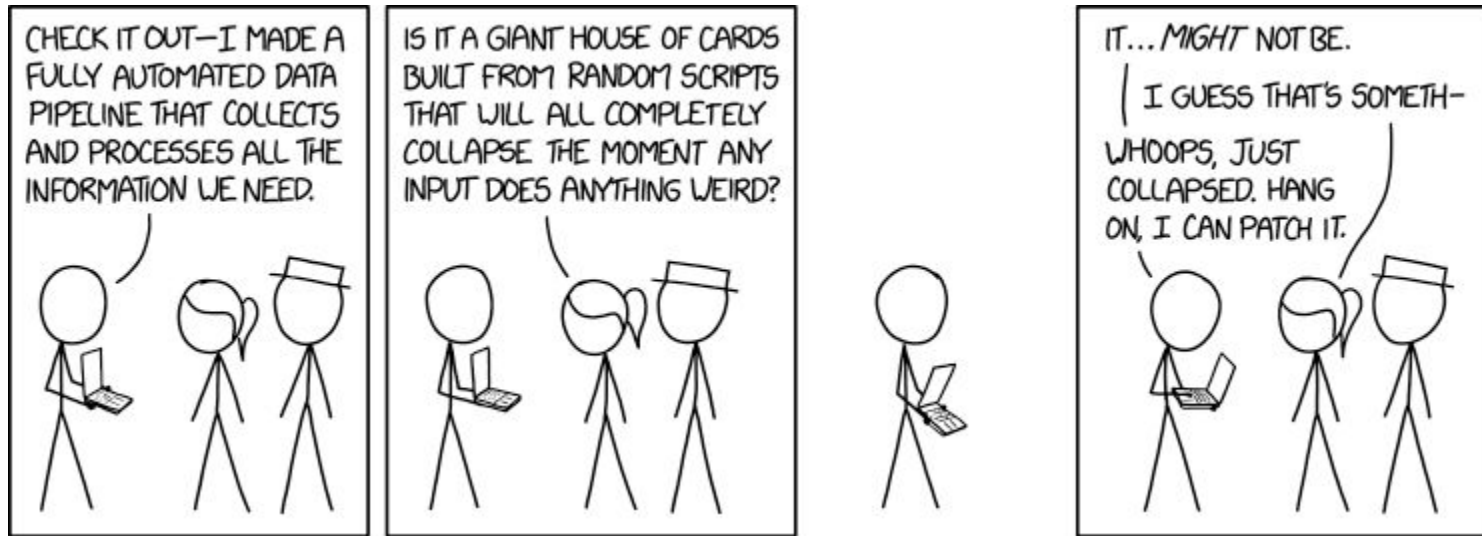


CSE 445: Machine Learning

Machine Learning Project – Data Exploration



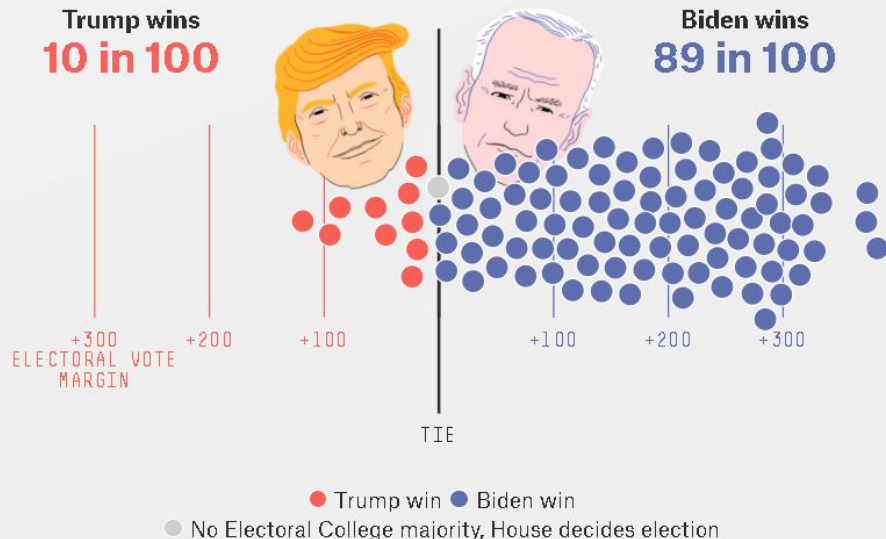
Steps in an ML Project

1. Look at the big picture
2. Get the data
3. Visualize data for insights
4. Preprocess the data
5. Select a model and train it
6. Fine-tune your model
7. Present solution
8. Launch!

Prediction gone wrong?

Biden is *avored* to win the election

We simulate the election 40,000 times to see who wins most often. The sample of 100 outcomes below gives you a good idea of the range of scenarios our model thinks is possible.



What if polls are as wrong as 2016? Biden still wins

State polling averages for Clinton pre-election and her final margin in that state. Current state polling averages for Biden, and what the margin would be if the 2016 errors were repeated

STATE	DEMOCRATS' POLLING LEAD OR DEFICIT			
	CLINTON 2016		BIDEN 2020	
	POLLING AVERAGE	FINAL MARGIN	POLLING AVERAGE	WITH THE 2016 ERROR ...
Arizona	-2.3	-3.5	+3.1	+1.9
Colorado	+3.8	+4.9	+13.8	+14.9
Florida	+0.5	-1.2	+2.1	+0.4
Georgia	-4.0	-5.1	+1.6	+0.5
Iowa	-3.4	-9.4	+0.1	-5.9
Maine	+6.9	-3.0	+13.8	+3.9
ME-2	-0.4	-10.3	+2.6	-7.3
Michigan	+4.0	-0.2	+8.8	+4.6
Minnesota	+5.9	+1.5	+8.9	+4.5
NE-2	-0.7	-2.2	+6.2	+4.7
Nevada	+0.7	+2.4	+5.8	+7.5
New Hampshire	+3.4	+0.4	+11.1	+8.1
New Mexico	+5.3	+8.2	+11.0	+13.9
North Carolina	+0.7	-3.7	+2.4	-2.0
Ohio	-2.0	-8.1	-0.9	-7.0
Pennsylvania	+3.7	-0.7	+5.1	+0.7
Texas	-8.5	-9.0	-1.0	-1.5
Virginia	+5.4	+5.3	+11.4	+11.3
Wisconsin	+5.4	-0.8	+8.6	+2.4

Source: fivethirtyeight

Example Problem: California Housing

- Objective: Use **census data** from California to **build a model of housing prices**
 - Source: StatLib repository

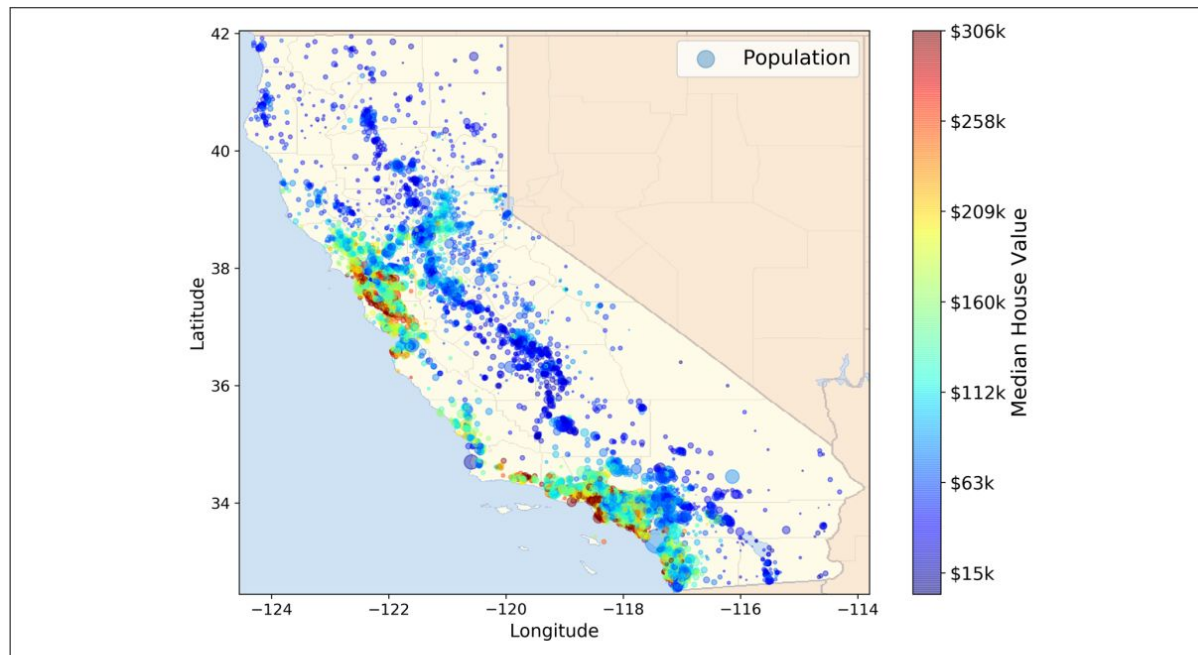
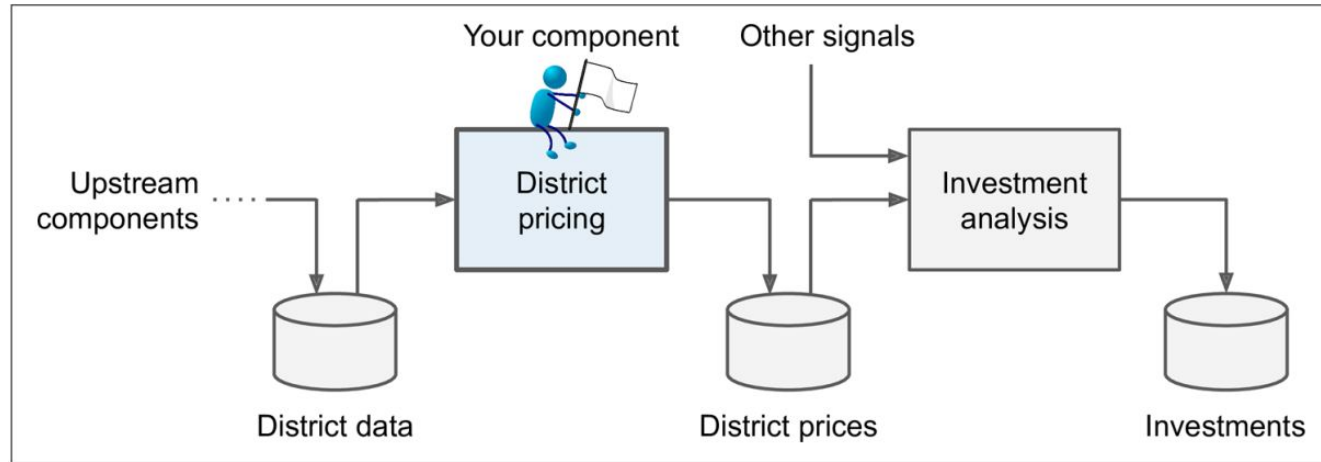


Figure 2-1. California housing prices

Frame the Problem

- Task: Predict median housing price in ANY district in CA, given all other metrics
 - Dataset includes metrics such as population, median household income, total rooms etc – as well as the median housing price in a district
 - Model output will go in as a signal to decide whether to invest in a given area



Data Pipeline
for District
pricing
project

Figure 2-2. A Machine Learning pipeline for real estate investments

What type of Problem is it?

- Supervised, unsupervised, reinforcement?
 - We have the **labeled** training examples – the median housing price is included in the dataset. We can use **Supervised Learning**!
- Classification or Regression?
 - The desired **output** is a **continuous** variable – this is a **regression** task
 - Multiple features available - it's a **multiple** regression problem
 - Only one value to be predicted – **univariate** regression problem
- Batch Learning or Online Learning?
 - No incoming data other than dataset – **batch** learning is fine

Performance Measure

- Root Mean Square Error (RMSE) or Mean Absolute Error (MAE)

Equation 2-1. Root Mean Square Error (RMSE)

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(\mathbf{x}^{(i)}) - y^{(i)} \right)^2}$$

Equation 2-2. Mean absolute error (MAE)

$$\text{MAE}(\mathbf{X}, h) = \frac{1}{m} \sum_{i=1}^m \left| h(\mathbf{x}^{(i)}) - y^{(i)} \right|$$

- m - number of **instances** in the dataset
- $\mathbf{x}^{(i)}$ – vector of all **feature** values for **instance i** and $y^{(i)}$ – **label** value for **instance i**
- $\mathbf{X}$ - matrix containing all feature values (excluding the labels) of all instances in dataset
- h – system's hypothesis/prediction function/weights
- $\text{RMSE}(\mathbf{X}, h)$ or $\text{MAE}(\mathbf{X}, h)$ – **cost function** measured on set of examples using your hypothesis h

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118.29 \\ 33.91 \\ 1,416 \\ 38,372 \end{pmatrix}$$

:

$$y^{(1)} = 156,400$$

Performance Measure

- Error – i.e. the difference between true label value y and the predicted label value computed as $h(\mathbf{x}^{(i)})$
- RMSE and MAE are two ways to measure the **distance** between two vectors – namely, the distance between your predicted labels, and the true label value
 - RMSE – straight line distance, or Euclidean norm, or L2 norm
 - MAE – city block distance, or Manhattan norm, or L1 norm
 - Higher norm index $\square$ more sensitive to outliers

Download Data

- Automate process of fetching data because
 - It's useful if the data changes regularly (can schedule this)
 - Can install on multiple machines

```
import os
import tarfile
import urllib
```

```
DOWNLOAD_ROOT = "https://raw.githubusercontent.com/ageron/handson-ml2/master/"
HOUSING_PATH = os.path.join("datasets", "housing")
HOUSING_URL = DOWNLOAD_ROOT + "datasets/housing/housing.tgz"
```

```
def fetch_housing_data(housing_url=HOUSING_URL, housing_path=HOUSING_PATH):
    os.makedirs(housing_path, exist_ok=True)
    tgz_path = os.path.join(housing_path, "housing.tgz")
    urllib.request.urlretrieve(housing_url, tgz_path)
    housing_tgz = tarfile.open(tgz_path)
    housing_tgz.extractall(path=housing_path)
    housing_tgz.close()
```

Quick Look at Data Structure

- Each row represents 1 district (i.e. 1 instance)
- Ten features (attributes)
- Total instances = 20,640 in the dataset

```
In [5]: housing = load_housing_data()  
housing.head()
```

```
Out[5]:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
0	-122.23	37.88	41.0	880.0	129.0	322.0
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0
2	-122.24	37.85	52.0	1467.0	190.0	496.0
3	-122.25	37.85	52.0	1274.0	235.0	558.0
4	-122.25	37.85	52.0	1627.0	280.0	565.0

Figure 2-5. Top five rows in the dataset

```
In [6]: housing.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 20640 entries, 0 to 20639  
Data columns (total 10 columns):  
longitude           20640 non-null float64  
latitude            20640 non-null float64  
housing_median_age  20640 non-null float64  
total_rooms         20640 non-null float64  
total_bedrooms      20433 non-null float64  
population          20640 non-null float64  
households          20640 non-null float64  
median_income       20640 non-null float64  
median_house_value  20640 non-null float64  
ocean_proximity     20640 non-null object  
dtypes: float64(9), object(1)  
memory usage: 1.6+ MB
```

Figure 2-6. Housing info

```
In [8]: housing.describe()
```

```
Out[8]:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553
std	2.003532	2.135952	12.585558	2181.615252	421.385070
min	-124.350000	32.540000	1.000000	2.000000	1.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000

Figure 2-7. Summary of each numerical attribute

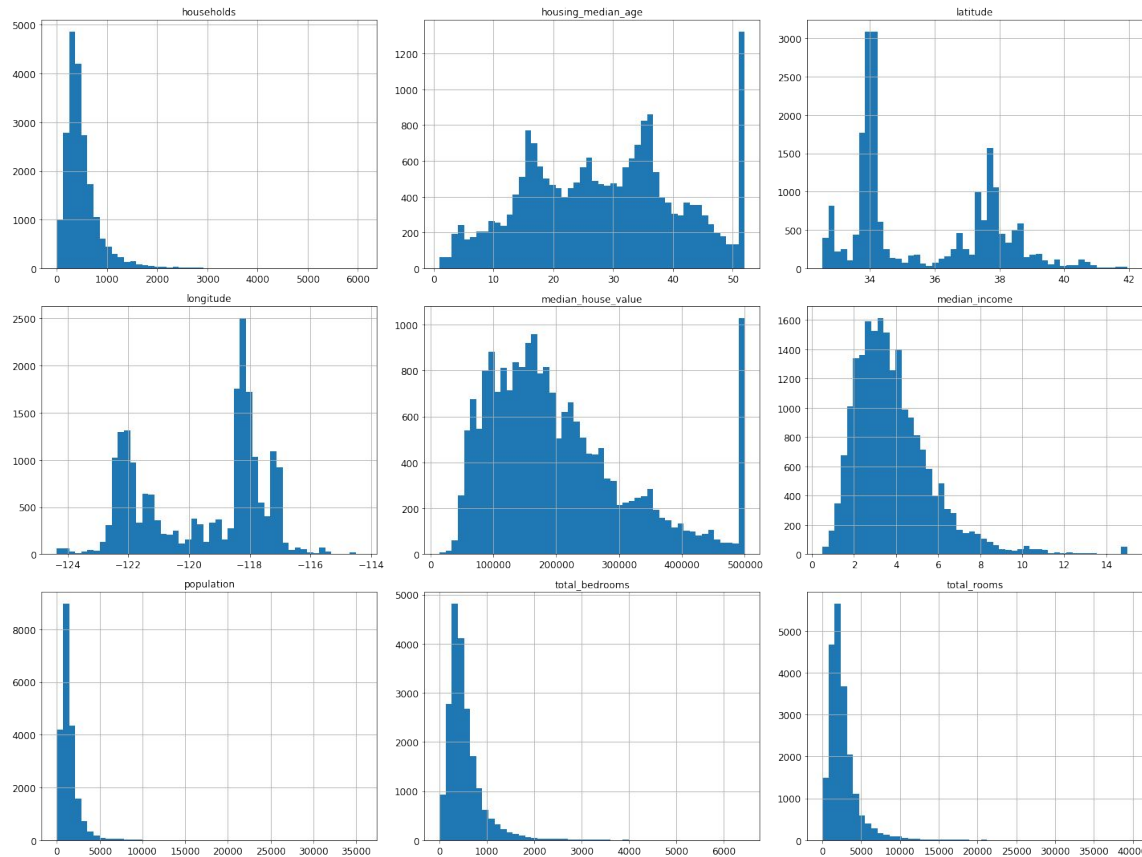
Histogram

- Plot a histogram for each numerical attribute

```
%matplotlib inline # only in a Jupyter notebook
import matplotlib.pyplot as plt
housing.hist(bins=50, figsize=(20,15))
plt.show()
```

- Noticeable oddities:

1. Preprocessed Features – median income isn't in raw USD, but has been scaled by a factor of 10,000 and capped to 15
2. Output (i.e. median house value) has also been capped – here go, will not be able to predict beyond \$500,000!
3. Tail-heavy histogram



Test Set Generation – Random Sampling

- Create and set aside the test set as early as possible! (Data Snooping bias)
- Use random seed generator to ensure same shuffled indices (if your dataset updates, this alone will not work!)
- Compute a hash of each instance and put in test set if it's less than or equal to 20% of max hash value (ensures consistency across multiple runs)

```
In [10]: # to make this notebook's output identical at every run
np.random.seed(42)
```

```
In [11]: import numpy as np

# For illustration only. Sklearn has train_test_split()
def split_train_test(data, test_ratio):
    shuffled_indices = np.random.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

```
In [12]: train_set, test_set = split_train_test(housing, 0.2)
print(len(train_set), "train +", len(test_set), "test")

16512 train + 4128 test
```

```
In [13]: from zlib import crc32

def test_set_check(identifier, test_ratio):
    return crc32(np.int64(identifier)) & 0xffffffff < test_ratio * 2**32

def split_train_test_by_id(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: test_set_check(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

```
In [16]: housing_with_id = housing.reset_index() # adds an `index` column
train_set, test_set = split_train_test_by_id(housing_with_id, 0.2, "index")
```

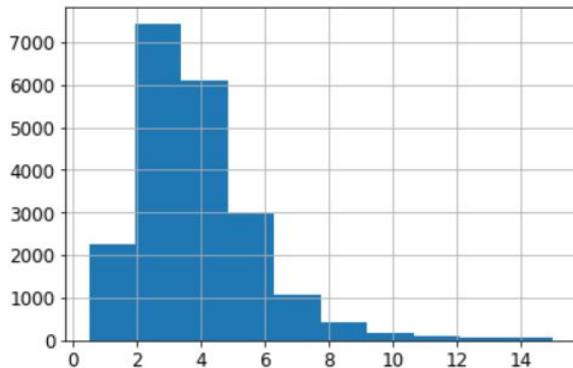
```
In [17]: housing_with_id["id"] = housing["longitude"] * 1000 + housing["latitude"]
train_set, test_set = split_train_test_by_id(housing_with_id, 0.2, "id")
```

Stratified Sampling

- Need to have a test set that is **representative** of the overall population
- **Strata** – homogenous subgroups
 - e.g. male to female ratio in Bangladesh is 103 to 100
- **Stratified sampling** – ensuring test set is representative of overall population
 - Test set should have 103 males for every 100 females

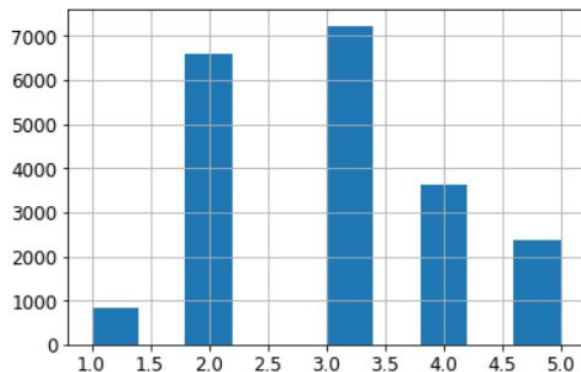
```
In [21]: housing["median_income"].hist()
```

```
Out[21]: <matplotlib.axes._subplots.AxesSubplot at 0x114f7c828>
```



```
In [24]: housing["income_cat"].hist()
```

```
Out[24]: <matplotlib.axes._subplots.AxesSubplot at 0x12b945588>
```



Stratified Sampling

- Without Stratified Sampling, the sampling bias in the test set is generally much higher
- We want the test set data to resemble the real world data

```
In [25]: from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
for train_index, test_index in split.split(housing, housing["income_cat"]):
    strat_train_set = housing.loc[train_index]
    strat_test_set = housing.loc[test_index]
```

```
In [26]: strat_test_set["income_cat"].value_counts() / len(strat_test_set)
```

```
Out[26]: 3    0.350533
         2    0.318798
         4    0.176357
         5    0.114583
         1    0.039729
         Name: income_cat, dtype: float64
```

	Overall	Stratified	Random	Rand. %error	Strat. %error
1	0.039826	0.039729	0.040213	0.973236	-0.243309
2	0.318847	0.318798	0.324370	1.732260	-0.015195
3	0.350581	0.350533	0.358527	2.266446	-0.013820
4	0.176308	0.176357	0.167393	-5.056334	0.027480
5	0.114438	0.114583	0.109496	-4.318374	0.127011

Figure 2-10. Sampling bias comparison of stratified versus purely random sampling

Data Visualization

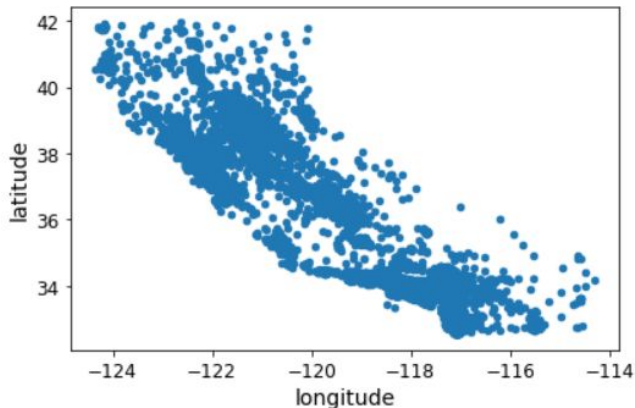
- Use latitude and longitude to visualize the district locations

Bad Visualization

```
In [31]: housing = strat_train_set.copy()
```

```
In [32]: housing.plot(kind="scatter", x="longitude", y="latitude")  
save_fig("bad_visualization_plot")
```

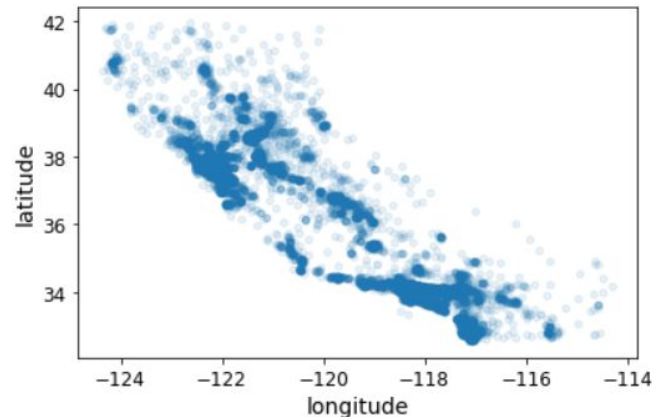
Saving figure bad_visualization_plot



Slightly Better Visualization

```
In [33]: housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.1)  
save_fig("better_visualization_plot")
```

Saving figure better_visualization_plot

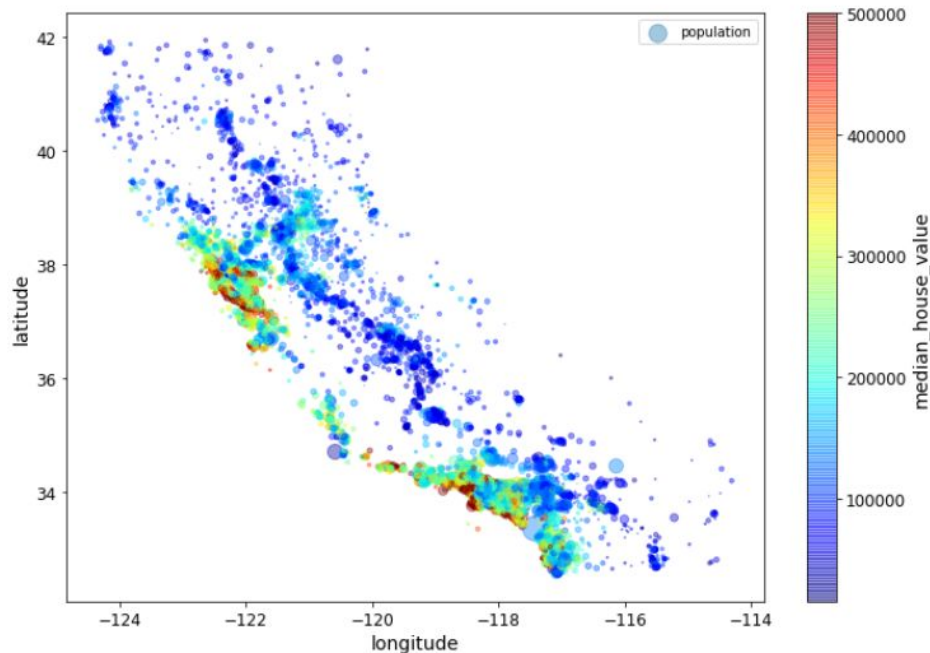


Data Visualization

- Best visualization – show median house value using color!
- Can see high value around metro areas near the ocean (LA, SF, SD etc)
- Note: Only do this for training set!

```
In [34]: housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.4,  
s=housing["population"]/100, label="population", figsize=(10,7),  
c="median_house_value", cmap=plt.get_cmap("jet"), colorbar=True,  
sharex=False)  
plt.legend()  
save_fig("housing_prices_scatterplot")
```

Saving figure housing_prices_scatterplot



Correlation between attributes

- Correlation coefficient near 1 ☐
Strong Positive Correlation
 - If x goes up, y goes up too
- Correlation coefficient near -1 ☐
Strong negative correlation
 - If x goes up, y goes down
- Correlation coefficient near 0 ☐ no **linear** correlation
 - Doesn't mean there's no relationship between the two!

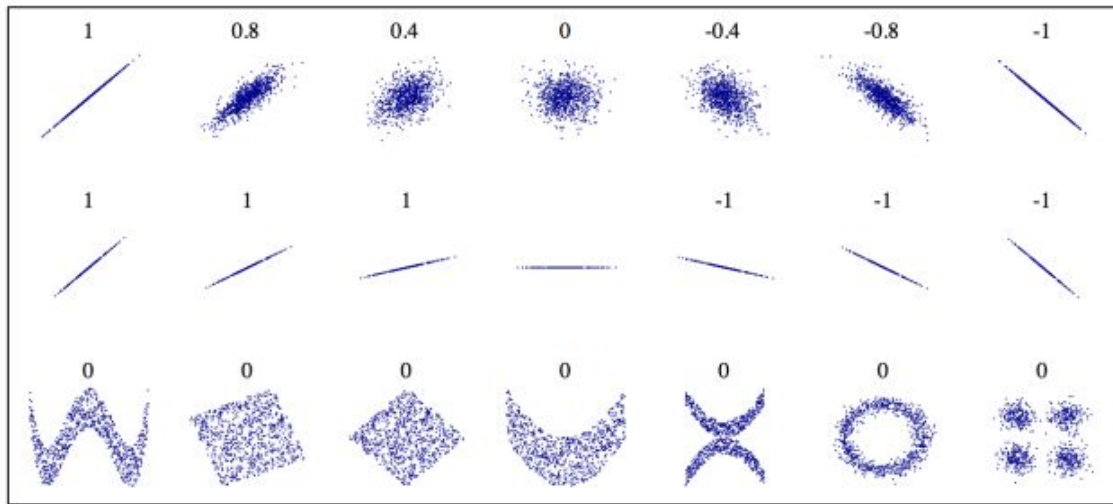


Figure 2-14. Standard correlation coefficient of various datasets (source: Wikipedia; public domain image)

```
In [36]: corr_matrix = housing.corr()
```

```
In [37]: corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
Out[37]: median_house_value    1.000000
median_income    0.687160
total_rooms      0.135097
housing_median_age 0.114110
households       0.064506
total_bedrooms   0.047689
population       -0.026920
longitude         -0.047432
latitude         -0.142724
Name: median_house_value, dtype: float64
```

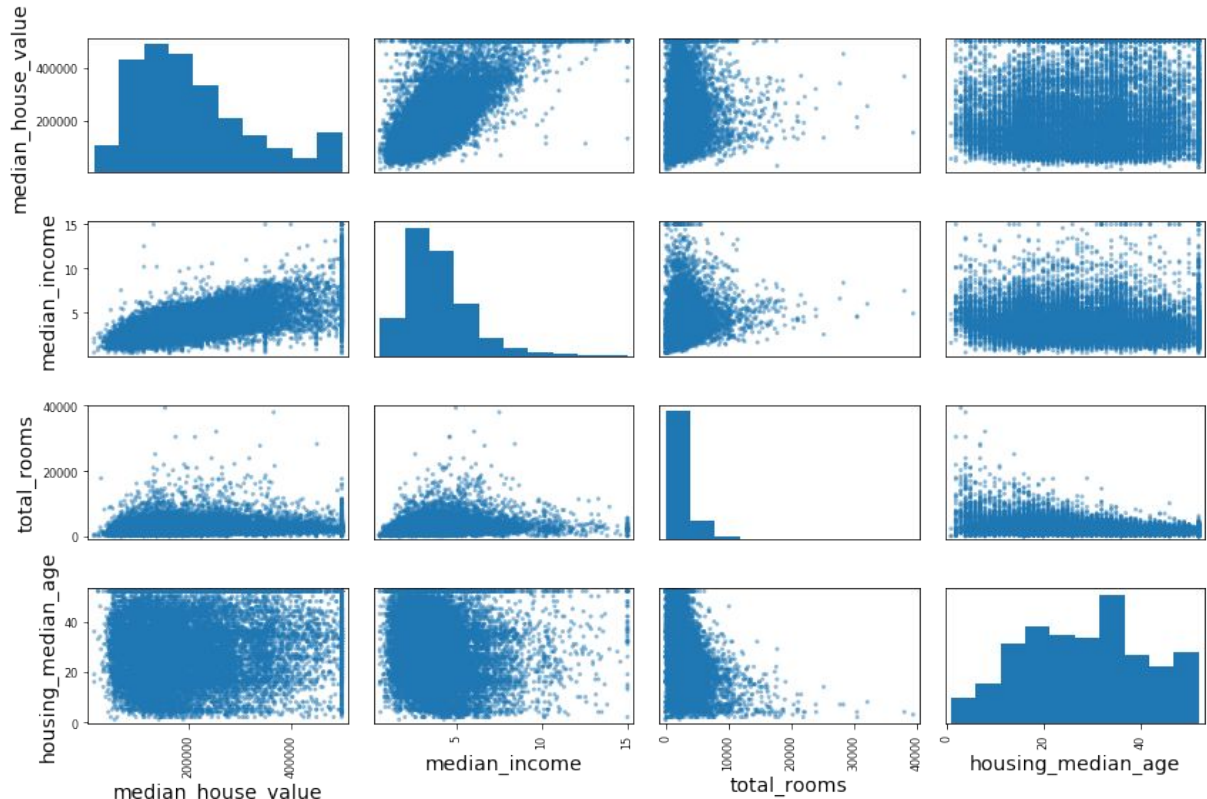
- Note: The size of the slope has nothing to do with how correlated the two attributes are!
- In case of housing dataset, median house value is positively correlated to median income (as expected)

Scatter matrix visualization of Correlation

- Main diagonal has each attributes' histogram generated by the pandas library (instead of a perfectly correlated straight line)
- Look for patterns in each attribute pair

Correlation coefficient with
median house value

median_house_value	1.000000
median_income	0.687160
total_rooms	0.135097
housing_median_age	0.114110
households	0.064506
total_bedrooms	0.047689
population	-0.026920
longitude	-0.047432
latitude	-0.142724



Attribute Combinations

- Upward trend visible – and the points are not too dispersed
 - Price cap visible at 500,000
 - Horizontal lines at 450,000, 350,000, and 280,000 (Why?)
- Algorithm may pick up and reproduce these quirks

```
housing["rooms_per_household"] = housing["total_rooms"]/housing["households"]  
housing["bedrooms_per_room"] = housing["total_bedrooms"]/housing["total_rooms"]  
housing["population_per_household"] = housing["population"]/housing["households"]
```

```
In [41]: corr_matrix = housing.corr()  
         corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
Out[41]: median_house_value    1.000000  
         median_income      0.687160  
         rooms_per_household  0.146285  
         total_rooms         0.135097  
         housing_median_age  0.114110  
         households          0.064506  
         total_bedrooms      0.047689  
         population_per_household -0.021985  
         population          -0.026920  
         longitude           -0.047432  
         latitude            -0.142724  
         bedrooms_per_room   -0.259984  
         Name: median_house_value, dtype: float64
```

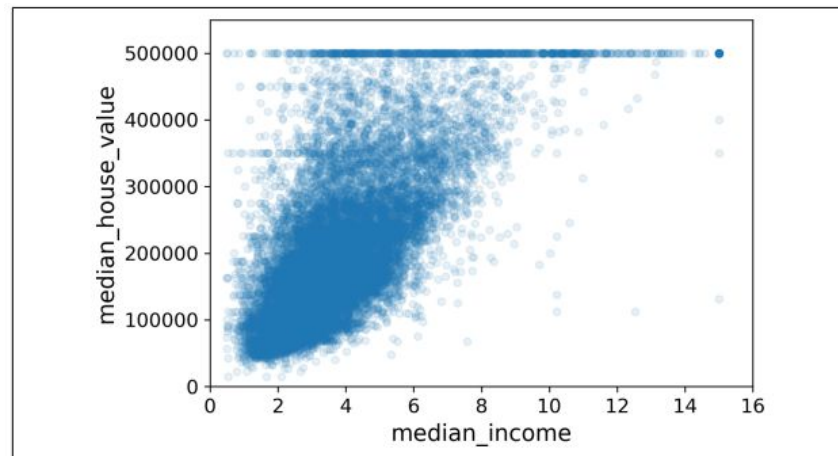


Figure 2-16. Median income versus median house value

- Consider other attribute combinations – may be more revealing!
 - Rooms per household, bedrooms per room, and population per household more correlated to house value than total rooms, households, or population alone!

Missing Data - Numerical

- Option 1: Remove instances (rows) that contain missing data [**dropna()** in pandas]
 - Can work if there aren't too many instances with missing data
- Option 2: Remove the entire attribute (column) with missing data [**drop()** in pandas]
 - Worse than option 1, especially if it's a useful attribute when available!
- Option 3: Set values to some statistical measure (zero, mean, median, etc) [**fillna()** in pandas]
 - Only do this on training set – replace later in test set during evaluation
 - Use SimpleImputer in Scikit-Learn to streamline this process

	name	gender	height	weight	age
0	Michael	None	123.0	10.0	14.0
1	Jessica	F	145.0	NaN	NaN
2	Sue	NaN	100.0	30.0	29.0
3	Jake	F	NaN	NaN	NaN
4	Amy	NaN	NaN	NaN	52.0
5	Tye	M	150.0	20.0	45.0

```
housing.dropna(subset=["total_bedrooms"])    # option 1
housing.drop("total_bedrooms", axis=1)       # option 2
median = housing["total_bedrooms"].median()  # option 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Text and Categorical Attributes

- Discrete, non-arbitrary text values – **category**
- Convert categorical values from text to numbers
- Option 1: Map each category to a unique number
 - ML algorithm may assume (incorrectly) that two “nearby” values are more similar than two distant values
 - <1H OCEAN and NEAR OCEAN encoded as 0 and 4 – even though they’re more similar to each other than INLAND (encoded to 1)
 - Only use this if the categories are **ordered**

```
>>> ordinal_encoder.categories_  
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],  
      dtype=object)]
```

```
>>> housing_cat = housing[["ocean_proximity"]]  
>>> housing_cat.head(10)
```

	ocean_proximity
17606	<1H OCEAN
18632	<1H OCEAN
14650	NEAR OCEAN
3230	INLAND
3555	<1H OCEAN
19480	INLAND
8879	<1H OCEAN
13685	INLAND
4937	<1H OCEAN
4861	<1H OCEAN

```
>>> from sklearn.preprocessing import OrdinalEncoder  
>>> ordinal_encoder = OrdinalEncoder()  
>>> housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)  
>>> housing_cat_encoded[:10]  
array([[0.],  
       [0.],  
       [4.],  
       [1.],  
       [0.],  
       [1.],  
       [0.],  
       [1.],  
       [0.],  
       [0.]])
```

Text and Categorical Attributes

- Option 2: One hot encoding – one dummy binary attribute per category
 - Sparse matrix with nonzero element location generated
 - May degrade performance if there are too many categories – consider replacing with useful numeric features related to categories if possible

```
>>> from sklearn.preprocessing import OneHotEncoder
>>> cat_encoder = OneHotEncoder()
>>> housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
>>> housing_cat_1hot
<16512x5 sparse matrix of type '<class 'numpy.float64'>'
  with 16512 stored elements in Compressed Sparse Row format>

>>> cat_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
      dtype=object)]
```

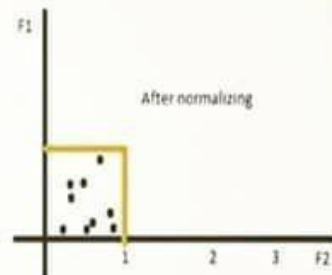
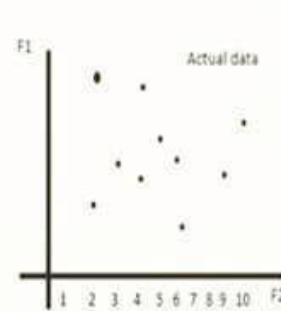
```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(10)
```

	ocean_proximity
17606	<1H OCEAN
18632	<1H OCEAN
14650	NEAR OCEAN
3230	INLAND
3555	<1H OCEAN
19480	INLAND
8879	<1H OCEAN
13685	INLAND
4937	<1H OCEAN
4861	<1H OCEAN

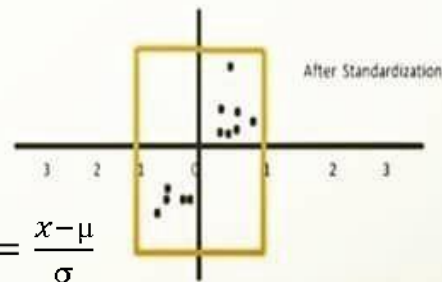
```
>>> housing_cat_1hot.toarray()
array([[1., 0., 0., 0., 0.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.],
       ...,
       [0., 1., 0., 0., 0.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```


Feature Scaling

- Usually, ML systems don't perform well if input attributes have very different scales
 - In housing data, total # of rooms $\square$ 6 to 39,320, whereas Median Income $\square$ 0 to 15
- Normalization $\square$ shift and rescale values so they range from 0 (minimum value) and 1 (maximum value)
 - Might be adversely affected by outliers
- Standardization $\square$ subtract the mean value, and divide by standard deviation so that the resulting distribution has zero mean, and unit variance
 - Not bound to a specific range (could be problematic for some algorithms such as NNs)



$$\text{Normalization: } x = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$



$$\text{Standardization: } x = \frac{x - \mu}{\sigma}$$

Custom Transformers

- Scikit-Learn relies on duck typing ☐ create a class and implement three methods: `fit()`, `transform()`, and `fit_transform()`
 - just add `TransformerMixin` as a base class to get `fit_transform()`
 - Adding `BaseEstimator` gives you two extra methods: `get_params()` and `set_params()`
- Consider the following ☐ add the custom parameter `bedrooms_per_room` to the set of input attributes
- One hyperparameter in the form of `add_bedrooms_per_room`

```
from sklearn.base import BaseEstimator, TransformerMixin

rooms_ix, bedrooms_ix, population_ix, households_ix = 3, 4, 5, 6


class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
    def __init__(self, add_bedrooms_per_room = True): # no *args or **kwargs
        self.add_bedrooms_per_room = add_bedrooms_per_room
    def fit(self, X, y=None):
        return self # nothing else to do
    def transform(self, X):
        rooms_per_household = X[:, rooms_ix] / X[:, households_ix]
        population_per_household = X[:, population_ix] / X[:, households_ix]
        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, bedrooms_ix] / X[:, rooms_ix]
            return np.c_[X, rooms_per_household, population_per_household,
                          bedrooms_per_room]
        else:
            return np.c_[X, rooms_per_household, population_per_household]

attr_adder = CombinedAttributesAdder(add_bedrooms_per_room=False)
housing_extra_attribs = attr_adder.transform(housing.values)
```


Pipeline

- Ensure data transformation steps are executed in the right order
- Pipeline constructor `>` take list of name/estimator pairs in sequence
- Column transformer `>` handle numerical and categorical data together
- `housing_prepared` contains data ready for training

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('attrs_adder', CombinedAttributesAdder()),
    ('std_scaler', StandardScaler()),
])

housing_num_tr = num_pipeline.fit_transform(housing_num)

from sklearn.compose import ColumnTransformer

num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(), cat_attribs),
])

housing_prepared = full_pipeline.fit_transform(housing)
```

Training – Linear Regression

```
from sklearn.linear_model import LinearRegression
```

```
lin_reg = LinearRegression()  
lin_reg.fit(housing_prepared, housing_labels)
```

```
>>> some_data = housing.iloc[:5]  
>>> some_labels = housing_labels.iloc[:5]  
>>> some_data_prepared = full_pipeline.transform(some_data)  
>>> print("Predictions:", lin_reg.predict(some_data_prepared))  
Predictions: [ 210644.6045  317768.8069  210956.4333  59218.9888  189747.5584]  
>>> print("Labels:", list(some_labels))  
Labels: [286600.0, 340600.0, 196900.0, 46300.0, 254500.0]
```

```
>>> from sklearn.metrics import mean_squared_error  
>>> housing_predictions = lin_reg.predict(housing_prepared)  
>>> lin_mse = mean_squared_error(housing_labels, housing_predictions)  
>>> lin_rmse = np.sqrt(lin_mse)  
>>> lin_rmse  
68628.19819848922
```

- Import LinearRegression and fit to prepared training data and corresponding labels
- Predictions are not exactly accurate
- RMSE is \$68,628 – better than nothing, but not great!
- Model is underfitting the data – model isn't powerful enough/need better features/reduce constraints

Training – Decision Tree Regression

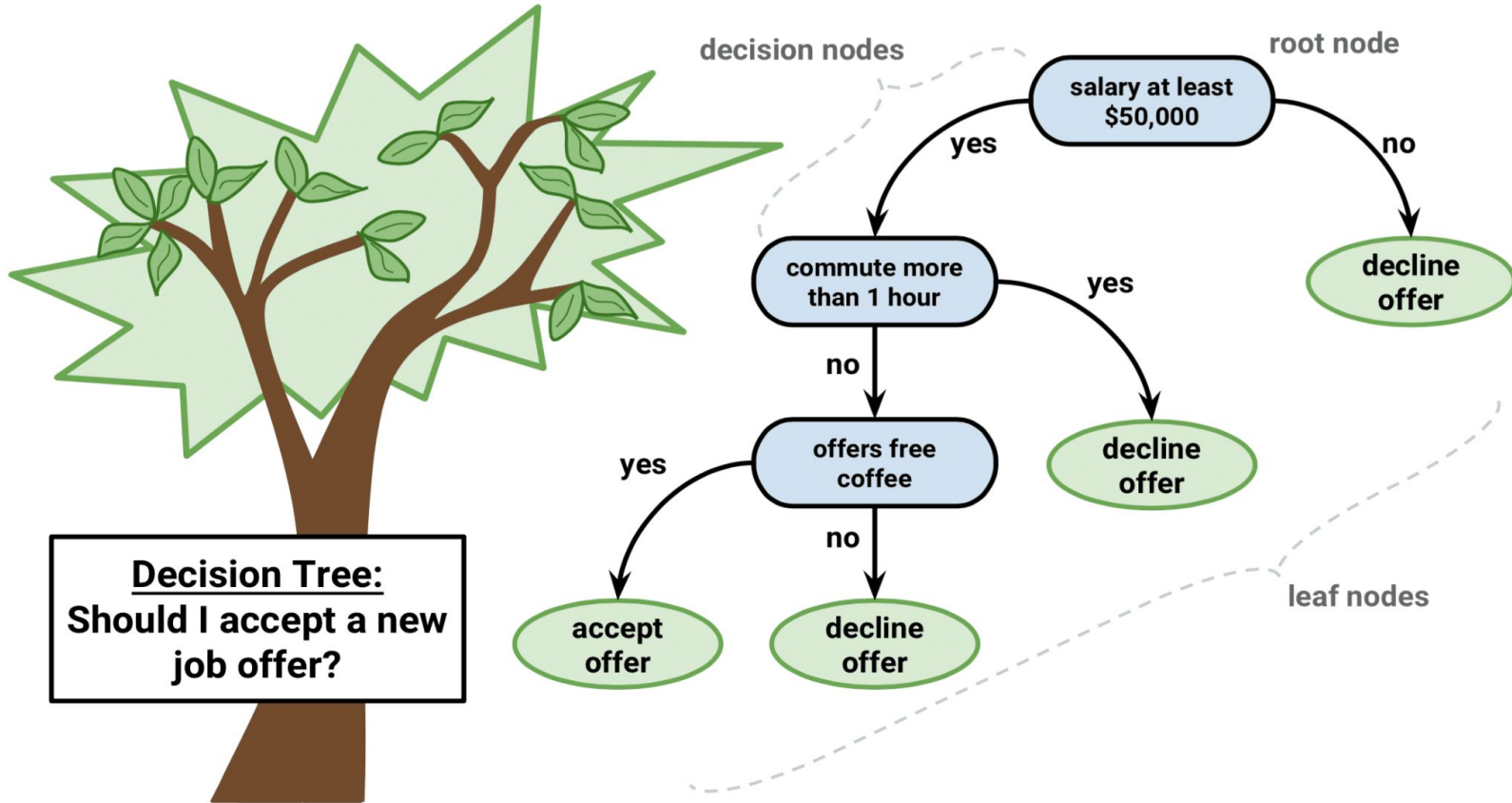
- Decision Trees □ more powerful than Linear Regression
- DTR can find complex, nonlinear relationships in the data – something linear regression is not capable of doing by definition
- No error in between housing labels and housing predictions?
- Tendency to overfit the data – need model validation!
- Note: Decision Trees often look great during the training phase, and fail miserably in validation/testing. Don't jump to conclusions too quickly!

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor()
tree_reg.fit(housing_prepared, housing_labels)

>>> housing_predictions = tree_reg.predict(housing_prepared)
>>> tree_mse = mean_squared_error(housing_labels, housing_predictions)
>>> tree_rmse = np.sqrt(tree_mse)
>>> tree_rmse
0.0
```

Training – Decision Tree Regression



Validation

■ Option 1: Single Validation set

- Use `train_test_split` to divide training set into training+validation
- Train your model with the smaller training set, and evaluate them against validation set

■ Option 2: K-fold cross-validation

- Randomly split training set into K subsets, or folds - train and evaluate model K times, picking one fold for eval and training on the rest
- Better to do this, especially if you have a small dataset

- Lin Reg model actually does better than the Decision Tree model!

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(tree_reg, housing_prepared, housing_labels,
                          scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-scores)

>>> def display_scores(scores):
...     print("Scores:", scores)
...     print("Mean:", scores.mean())
...     print("Standard deviation:", scores.std())
...
>>> display_scores(tree_rmse_scores)
Scores: [70194.33680785 66855.16363941 72432.58244769 70758.73896782
 71115.88230639 75585.14172901 70262.86139133 70273.6325285
 75366.87952553 71231.65726027]
Mean: 71407.68766037929
Standard deviation: 2439.4345041191004

>>> lin_scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
...                               scoring="neg_mean_squared_error", cv=10)
...
>>> lin_rmse_scores = np.sqrt(-lin_scores)
>>> display_scores(lin_rmse_scores)
Scores: [66782.73843989 66960.118071 70347.95244419 74739.57052552
 68031.13388938 71193.84183426 64969.63056405 68281.61137997
 71552.91566558 67665.10082067]
Mean: 69052.46136345083
Standard deviation: 2731.674001798348
```

Fine Tuning Models

- Grid Search
 - Specify hyperparameters to experiment with
 - Use cross validation to evaluate all possible combinations
- Randomized Search
 - Useful when hyperparameter search space is large
 - More control over computing budget – set number of iterations
- Ensemble Methods
 - Group of models often work better than a single individual model
- Analyze the best models (and features) and their errors – then predict on the test set

Evaluation on Test Set

- Get predictors and labels from test set, set into pipeline, and evaluate!
 - Performance may be (and often is) worse than what was measured in cross validation
 - Final performance of system for housing data is comparable to human experts (but not better – still worth it!)

```
final_model = grid_search.best_estimator_  
  
X_test = strat_test_set.drop("median_house_value", axis=1)  
y_test = strat_test_set["median_house_value"].copy()  
  
X_test_prepared = full_pipeline.transform(X_test)  
final_predictions = final_model.predict(X_test_prepared)  
  
final_mse = mean_squared_error(y_test, final_predictions)  
final_rmse = np.sqrt(final_mse)    # => evaluates to 47,730.2
```


Check your Bias

- During WW2, Allied Forces ran an analysis on airplanes that took bullets during combat
- They wanted to figure out where planes were most vulnerable and needed armor
- The red dots are bullet holes observed in planes that returned from combat
- Where would YOU increase armor?

